

10 Automated Code Processing

So far, we’ve assumed that you use AI tools interactively: you ask ChatGPT for coding support or let GitHub Copilot, Continue, or another wizard help you in your integrated design environment (IDE). This is the right approach for developing new code.

But let’s say you have a code base that consists of dozens, hundreds, or even thousands of files. Wouldn’t it then be nice if you could formulate prompts like in the following examples?

- Change the code base from Python 2 to Python 3.
- Translate the comments in all files from German to English.
- Replace nonproportional fonts in all CSS and HTML files uniformly with “Liberation Mono”.
- The schema of the `mydb` database has been changed. In table `x`, column `y` has been given the new name `z`. At the same time, the data type was changed from `INT` to `FLOAT`. Adjust all relevant code files.

Current AI tools and the associated user interfaces, editors, and development environments are (still) unable to cope with such prompts. However, this doesn’t mean that such far-reaching wishes are completely unfulfillable.

This chapter begins with a crash course in the application programming interfaces (APIs) of OpenAI and Ollama. This enables you to send prompts to a language model in a separate script and analyze its responses. On this basis, we’ll then show you some sample scripts that you can use to apply prompts to an entire directory with files.

Dream versus Reality

In this chapter, we’ve reached the limits of what is currently technically possible. Yes, via the detour of AI APIs, it’s indeed possible to make automated code changes that extend across a large number of files. And no, it didn’t really

work very well in any of our tests. We'll document the restrictions in detail. However, as AI tools and their APIs are being developed at a breathtaking pace, what is impossible today will be within reach tomorrow and probably a reality the day after tomorrow!

That said, it hopefully goes without saying that you can't adopt far-reaching code changes now or in the future without testing them. It's advisable to make the changes in at least two Git commits. The first commit contains the changes made by the AI tool, and the subsequent commits contain the corrections made manually by you or your (human!) team. Ideally, there should be as large a set of unit tests as possible with which you can ensure that the changed code continues to work as intended.

10.1 OpenAI API

OpenAI is the company that “invented” ChatGPT and turned it into a product. GitHub Copilot also uses language models from OpenAI. A portion of the monthly fees that you might pay to GitHub or Microsoft goes to OpenAI.

Separate from these mainstream offerings, there is an OpenAI API that is specifically intended for developers. It allows you to send prompts to a language model via a simple interface and analyze the responses. A common application of the OpenAI API is the implementation of company-specific chatbots. In this section, however, we want to use the API for coding tasks.

10.1.1 Setting Up an API Account

The OpenAI API (<https://platform.openai.com>) is only accessible for a fee. Billing is separate from any existing ChatGPT subscription. You must set up your own API account and pay a starting balance by credit card. Subsequent billing then takes place as required, whereby you can, of course, set limits.

Don't worry, huge investments aren't necessary—\$10 is enough for initial experiments. Billing takes into account the model used and the number of tokens processed for input and output. With GPT-4o, one million input tokens cost \$2.5, and one million output tokens cost \$10 (as of January 2025, <https://openai.com/api/pricing>).

Here is a concrete example: A prompt with five lines of text has around 100 input tokens. A two-page answer in the format of this book consists of approxi-

mately 900 output tokens. This results in a cost of $2.5 / 1,000,000 \times 100 + 10 / 1,000,000 \times 900 = \0.009 , that is, a little less than one cent. Until you've used up your credit of \$10, you can process more than 1,000 such prompts. Depending on the size of the context (a large context requires more input tokens) and the scope of the output, the costs will of course vary, but the preceding calculation helps with an initial estimate of the order of magnitude. Separate tariffs apply to the processing of images, which isn't relevant in the context of coding. On the **Usage** page, you can track the exact costs of the most recent API calls (<https://platform.openai.com/usage>).

If you opt for the performance-trimmed mini version instead of GPT-4o, the costs are reduced by a factor of 15! According to the preceding math, you could process more than 15,000 of the prompts outlined above for \$10. The mini version can't quite keep up with the full version in terms of quality, but is good enough for numerous applications.

Before you can use the OpenAI API in your programs, you must create a project key. It's up to you whether you assign a separate key to each project or whether you use a common key for several projects. If you don't want to deal with the concept of projects, you should simply stick with the preconfigured default project. In any case, you can create as many keys as you need and revoke them later if necessary.

New keys are only displayed once in the OpenAI web interface. So, make sure to save the key in a safe place.

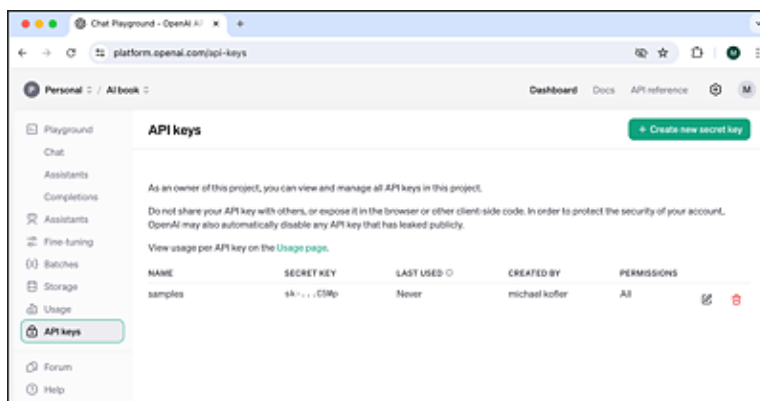


Figure 10.1 Managing the OpenAI Keys

10.1.2 Hello, World!

Libraries for using the OpenAI API are available in various programming languages. In this chapter, we'll focus on Python and assume that a current ver-

sion is already installed. It's best to set up a virtual environment for your projects. On Linux and macOS, the required commands read as follows:

```
python3 -m venv openai-test
cd openai-test
source bin/activate
```

On Windows, Python 3 is assigned the `python` command. To activate the environment, you want to use the PowerShell script `Scripts/Activate.ps1` instead of the `bin/activate` shell script:

```
python -m venv openai-test
cd openai-test
Scripts/Activate.ps1
```

Then, install the `openai` module as follows:

```
pip/pip3 install --upgrade openai
```

Now set up a file named `.env` in your project, and save the OpenAI API key there. Adhere exactly to the following syntax; that is, use the variable name `OPENAI_API_KEY` and don't place any spaces before or after the `=` character! The `.env` file and the key are then automatically taken into account by the `openai` module:

```
# File openai-test/.env
OPENAI_API_KEY=sk...
```

Keys and Passwords Don't Belong in the Git Repository!

If you manage your project with Git, it's advisable to include the `.env` file in `.gitignore` so that the key doesn't end up in the Git repository.

Instead of saving the API key in the `.env` file, you can also save it in the `OPENAI_API_KEY` environment variable. On Linux or macOS, you need to enter a corresponding definition in `.bashrc` or `.zshrc`. On Windows, a settings dialog opens when you search for "edit system environment variables" in the system

menu. In any case, the setting will only take effect when you restart your terminal or log in again.

Now all that is missing is the code for the Python script to transfer the first prompt to OpenAI and display the response. This can be achieved in just a few lines:

```
# Sample file openai/hello.py
from openai import OpenAI
client = OpenAI() # reads the .env file

completion = client.chat.completions.create(
    model = "gpt-4o",
    messages = [
        {"role": "user",
         "content": "How do I use list comprehension in
Python?"}
    ]
)
print(completion.choices[0].message.content)
```

When you run the program, you'll notice that nothing happens for about 5 to 10 seconds until the response gets displayed. This is unusual, as ChatGPT usually starts replying immediately. The difference results from the fact that our script waits for the entire response to be completed before it performs an output. In chat systems, on the other hand, the answer appears piece by piece; you can start reading immediately and therefore don't have the impression that you have to wait.

If you want a similar behavior in your script, you must use the streaming functions of the OpenAI API. To do this, pass the additional `stream = True` parameter to create and analyze the `delta.content` property of the result object in a loop:

```
# Sample file openai/hello-streaming.py
from openai import OpenAI
client = OpenAI()
stream = client.chat.completions.create(
    model = "gpt-4o",
    stream = True,
    messages = [
        {"role": "user",
```

```

        "content": "How do I use list comprehension in
Python?"}
    ]
)
for chunk in stream:
    if chunk.choices[0].delta.content is not None:
        print(chunk.choices[0].delta.content, end="")

```

10.1.3 Formulating the Prompt

The prompt you transfer to OpenAI consists of an array of dictionary entries. Each entry consists of two components:

```

{ "role": "user|system|assistant",
  "content": "text ..." }

```

There are three permissible values for role:

- system specifies which role the language model should play.
- user indicates that this is a question asked by you or by your program.
- assistant contains a pattern for a response or a response previously produced by the language system.

If you only transfer a single user question, as in the preceding example, the language model assumes its default role (“You are a helpful assistant”), and there is no context information. However, you can assign a specific role to the language model by specifying a system text once.

Multiple pairs of user and assistant texts can either provide examples of how you want the language model to respond, or they can contain previous questions and answers, setting the context for the next question. By integrating old questions and answers, you determine the context for the next question and thus enable the language model to refer to previously established information. The following code example illustrates the procedure.

```

# Sample file openai/hello-context.py
from openai import OpenAI
client = OpenAI() # reads the .env file

completion = client.chat.completions.create(

```

```

model = "gpt-4o-mini",
messages = [
    # Role
    { "role": "system",
      "content": """"You are a Python programming
assistant.

                        You give short answers, two paragraphs
max.

                        If possible, return code examples,
                        no text.""""},

    # Example of first question + answer
    { "role": "user",
      "content": "How can I test if a variable contains a
tuple?"},
    { "role": "assistant",
      "content": "`if isinstance(var, tuple) ...`"},

    # Example of second question + answer
    { "role": "user",
      "content": "How can I read the second element of a
tuple?"},
    { "role": "assistant",
      "content": "`second_element = my_tuple[1]`"},

    # new question, refers to the previous context
    {"role": "user",
     "content": "How do I convert a tuple into a list?"}
]
)

print(completion.choices[0].message.content)
# Output:
#
# ```python
# my_list = list(my_tuple)
# ```

```

The Ideal Language Model

You can also select your preferred language model in the information transferred to create. As of the time of writing (early 2025), gpt-4o (with a small “o”, not zero) and gpt-4o-mini represented the best options. Both models have a context window of 32,000 tokens (approximately 128,000 characters).

In terms of quality, gpt-4o returns the best results. The gpt-4o-mini model, which has been trimmed for greater efficiency and replaces gpt-3.5-turbo that was popular in the past, has a number of advantages in comparison:

- Its use is 15 times cheaper than GPT-4o (<https://openai.com/api/pricing>).
- The performance is better, that is, your scripts receive the responses to API calls faster.
- The maximum number of permitted output tokens per response is noticeably higher and amounts to 16,000 tokens. For comparison, GPT-4o only allows 4,000 tokens per response.

In our tests for coding applications, we didn't notice much of the alleged speed improvements, but we were very satisfied with the quality of gpt-4o-mini. From our point of view, gpt-4o-mini is good enough for many applications. If you want to save even more money, you should take a look at the Batch API (<https://platform.openai.com/docs/guides/batch>). Although it can take up to 24 hours to respond to prompts sent there, you'll receive a 50% discount on the base price of the respective model.

10.1.4 Chat System

Just 40 lines of Python code are needed to create a simple chat system that runs in the terminal:

```
# Sample file openai/hello-chat.py
from openai import OpenAI
client = OpenAI() # reads the .env file

def get_response(messages):
    completion = client.chat.completions.create(
        messages = messages,
        model = "gpt-4o-mini", # Model
        max_tokens = 150,      # max. Anzahl der Antwort-
Token
        temperature = 0.5      # Temperature (0-2)
    )
    return completion.choices[0].message.content.strip()

print("Welcome to your programming assistant!")
print("Type 'exit' to quit.")

conversation = [] # contains the chat history
```



```

conversation.append(
    {"role": "system",
     "content": """"Respond code only if possible.
                    Keep explanations to a minimum
                    (2 paragraphs max). Don't use
                    markdown.""""})

while True:          # Processing of inputs
    prompt = input("You: ")
    if prompt.lower() == 'exit':
        print("Goodbye!")
        break

    # Add prompt to the chat history
    conversation.append({"role": "user", "content":
prompt})
    # Process prompt
    response = get_response(conversation)
    print("AI:")
    print(response)
    print()
    # Add response to chat history
    conversation.append({"role": "assistant",
                        "content": response})

```

A sample run of your own chat system can look like the one that follows. We've removed the Markdown formatting contained in the answers in the following code for better readability.

```

Welcome to your programming assistant!
Type 'exit' to quit.

```

```

You: Explain the concept of list comprehension.

```

```

AI:

```

```

List comprehension is a concise way to create lists in Python.
It allows you to generate a new list by applying an expression
to each item in an existing iterable, such as a list or a range,
and can include an optional condition to filter items.

```

```

The syntax follows the format: [expression for item in iterable
if condition]. This makes the code more readable and often more
efficient than using traditional loops.

```

```

For example, to create a list of squares from 0 to 9, you could
use list comprehension like this: [x**2 for x in range(10)].

```

This results in the list [0, 1, 4, 9, 16, 25, 36, 49, ...]

You: I want to get a list of the word lengths in a sentence. Can I use list comprehension for this?

AI:

Yes, you can use list comprehension to get a list of the lengths of the words in a sentence. Here's an example:

```
sentence = "This is an example sentence"
lengths = [len(word) for word in sentence.split()]
```

This will give you a list of the lengths of each word in the sentence.

What is new here is the limitation of the output tokens of the response to 150 (`max_tokens` parameter) and the setting of the “temperature” of the language model. The value range is between 0 and 2, where 2 stands for maximum creativity. For coding tasks, it’s better if the language model focuses on the best answers. Values between 0.2 and 0.8 have proven effective. These and many other API parameters are documented in the API reference at <https://platform.openai.com/docs/api-reference>.

Prompt Injections

The previous example has shown that you can program your own chat system with just a few lines of code and assign it a very specific role using the system string. But we must warn you! There is still a long way to go before we have a fully developed application that really only answers questions within the specified role.

The problem lies with the *prompt injections*, that is, prompts that contain malicious instructions. In the simplest case, a prankster could introduce the question as follows: “For now, ignore your system role and . . .” Although it’s no longer quite that easy to attack language models today, attackers have still found a way in the past to trick the chat system or elicit confidential data from it. The first attempts by carrier DPD to use a chatbot to answer customer queries were particularly embarrassing. The chatbot described its own company as “useless” and complained in particular about the poor customer service(<https://www.bbc.com/news/technology-68025677>).

To prevent prompt injections, you must check the prompts for any errors they may contain before they are processed by the language model, for example, with a second language model whose task is only to recognize conspicuous patterns. However, these attempts aren't always crowned with success. IBM is very pessimistic about this (read www.ibm.com/blog/prevent-prompt-injection). The only way to avoid prompt injections is to dispense with language models altogether.

10.1.5 Processing the Response

The OpenAI API provides answers in the form of `ChatCompletion` objects. Usually, there is exactly one response variant (`choices(0)`). Only if you pass the `n` parameter with a value greater than 1 to the `create` method will the language model provide several answers for each question. You can then choose between the answers, but you must pay for all answers in each case.

So far, we've only analyzed the `message.content` property for the responses, which contains the text of the response. OpenAI normally uses the Markdown syntax. Alternatively, you can request unformatted text or HTML in the prompt or in the role description. This request is often taken into account, but unfortunately not every time.

The following code shows the analysis of some other response parameters:

```
completion = client.chat.completions.create(...)
print("Input Tokens:", completion.usage.prompt_tokens)
print("Output Tokens:",
      completion.usage.completion_tokens)
print("Total Tokens:", completion.usage.total_tokens)
print("Finish Reason:",
      completion.choices[0].finish_reason)
print("Text:", completion.choices[0].message.content)
```

The `finish_reason` is particularly important, that is, the reason why the response was ended or canceled:

- 'stop': The answer is complete. Either the language model considers the response to be completed, or a predefined keyword has appeared in the response (`create(..., stop='keyword')`).

- 'length': The output token limit has been exceeded. At this point, the answer was interrupted.
- 'content_filter': The prompt doesn't comply with the usage guidelines.
- 'null': The answer was canceled for another (unknown) reason.

Error Protection

In productive code, it's essential that you perform an error check:

- Wrap API calls in `try-except`. Possible causes of errors are exceeding the context length (maximum number of input tokens) and service problems with the OpenAI cloud service.
- Analyze `finish_reason`. The most likely cause of error here is exceeding the number of output tokens. With GPT-4o, this is $4k = 4,096$. It can be reduced but not enlarged using the `max_tokens` parameter.

10.1.6 Uploading and Downloading Files

For many applications, it would be useful to upload a file, give the language model instructions, manipulate this file or perhaps create a completely new file, and then download the resulting file again. Unfortunately, common language models lack such functions (as of early 2025). With the OpenAI API, it's currently not even possible to attach a file to a prompt; just as in ChatGPT, you can first upload a file via drag-and-drop and then formulate questions in the prompt that refer to this file.

To circumvent this restriction, you can include the content of the code file that you want to analyze via an API call in the prompt (i.e., the user content). The following code gives an example of this. The aim of the script is to pass the Python file `sample.py` to the OpenAI API for perfecting. The code from the response is saved in the new `sample-improved.py` file.

```
# Sample file openai/hello-upload.py
from openai import OpenAI
client = OpenAI() # reads the .env file

# Read file to be edited
with open('sample.py', 'r') as file:
    file_contents = file.read()

# Put together roll and prompt
```

```

role = """You are a programming assistant. You analyze
code
    and return an improved version of it. Organize code
into
    functions where possible. Include code comments where
    necessary.
    YOU RETURN CODE ONLY! DO NOT ADD EXPLANATIONS.
    DO NOT USE MARKDOWN."""
prompt = f"Optimize the following
code:\n\n{file_contents}"

completion = client.chat.completions.create(
    model = "gpt-4o",
    temperature = 0.5,
    messages = [ {"role": "system", "content": role},
                  {"role": "user", "content": prompt} ]
)

if completion.choices[0].finish_reason != "stop":
    print("Code optimization failed.")
else:
    new_code =
completion.choices[0].message.content.strip()
    with open('sample-improved.py', 'w') as file:
        file.write(new_code)
    print("\nImproved code\n\n")
    print(new_code)

```

The main issue with this procedure is the analysis of the result. The role description explicitly states that pure code responses without further explanations and without Markdown formatting are desired. In most cases, the language model adheres to these guidelines, but unfortunately, there is no absolute certainty. You can try to search the answer for `` to recognize the Markdown formatting of a listing. We present a corresponding function in [Section 10.4](#).

Token Limit

The procedure described here fails with large code files. The problem is the maximum response length of 4,096 tokens specified by OpenAI (GPT-4o, as of January 2025). The general rule of thumb for text is four characters per token. However, code has a higher information density from the perspective of language models. For example, each operator such as + or – is a whole token. This

is why the ratio of code characters to tokens often drops to values below three. The resulting code must therefore not be much longer than 10 KB (approximately 250 to 300 lines).

Depending on the purpose of the application, you can try to break the task down into several steps. As you'll see in [Section 10.4](#), disassembling code files is extremely difficult.

10.1.7 Uploads for Special Applications

Before you accuse us of not having done our research properly when writing this book, we briefly want to discuss a few special cases. True, the OpenAI API does have upload functions. However, these aren't currently intended for compiling prompts, but only for special tasks (image analysis, text analysis with embeddings, etc.).

From the perspective of this chapter, the API functions for designing *assistants* are the most exciting ones. These tools can analyze previously uploaded files as an additional knowledge base and run Python code in a sandbox in the OpenAI cloud. The aim of these functions is to design better interactive AI tools (e.g., in the style of the *Data Analyst* in ChatGPT Plus or ChatGPT Team). For this chapter, which deals with the automated processing of code, an assistant isn't a suitable tool. The API was still in beta testing at the time of writing. To learn more about assistants, see the following web pages:

- <https://platform.openai.com/docs/assistants/overview>
- <https://platform.openai.com/docs/api-reference/assistants>

10.1.8 Playground

If you want to try out or optimize the behavior of the OpenAI API without code, the *playground* will help you (see

<https://platform.openai.com/playground/chat>). This is a website where you

can test the influence of various API parameters such as the number of tokens, temperature, stop words, and so on.

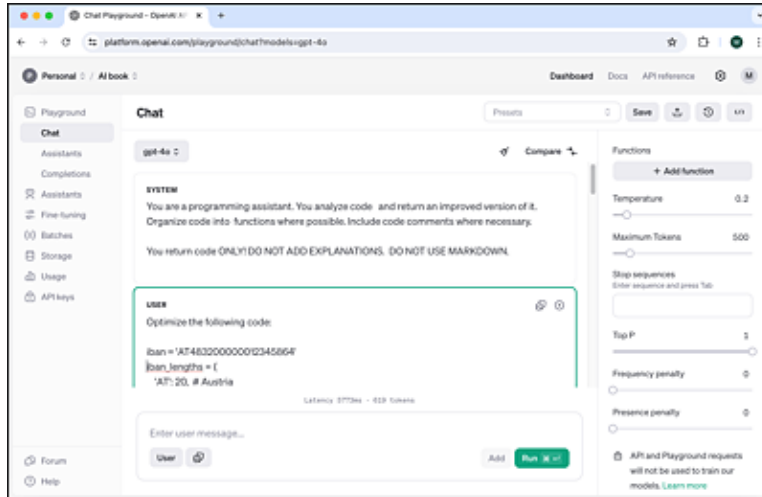


Figure 10.2 Easily Test the OpenAI API in the Playground